Intelligent System Development

Developing Multiagent Systems

A Practical Tutorial with Python and PyTorch

Level:	Senior undergraduate students
Topic:	Intelligent agents and multiagent systems
Programming:	Python, with optional PyTorch
Applications:	E-commerce, tourism, healthcare
Format:	Theory, implementation, laboratory exercises

Tutorial objectives
This tutorial introduces the principles of intelligent agents, multiagent coordination, software architecture, learned decision components, and application-oriented evaluation. Students will design and implement a small multiagent system and investigate how several specialized agents cooperate to solve a practical problem.

Teaching material

How to Use This Tutorial

The tutorial is organized into 20 teaching units. Each unit can be used as a lecture segment, laboratory handout, or independent reading assignment.

Learning outcomes

After completing the tutorial, students should be able to:

1. Explain intelligent agents, autonomy, goals, percepts, actions, and environments.
2. Identify applications that benefit from multiple agents.
3. Design agent roles, communication messages, and coordination policies.
4. Implement a basic multiagent system in Python.
5. Integrate a trained PyTorch model into an agent.
6. Evaluate quality, latency, robustness, and safety.
7. Propose a multiagent solution for e-commerce, tourism, or healthcare support.

Suggested teaching schedule

Time	Activity
45 minutes	Concepts and examples
30 minutes	Architecture and agent design
60 minutes	Python implementation
30 minutes	Testing and application discussion
15 minutes	Review and assignment briefing

The implementation begins with deterministic agents so that students can understand coordination before adding machine learning. The PyTorch section is an extension, not a substitute for the complete system design.

1 What Is a Multiagent System?

An **intelligent agent** is a computational entity that perceives its environment and selects actions to achieve objectives. A **multiagent system** (MAS) consists of multiple agents whose actions interact through messages, shared state, or a common environment.

Examples include shopping assistants, autonomous vehicles, traffic-control systems, robotic teams, and distributed scheduling applications.

1.1 From an AI model to an intelligent system

A machine-learning model may predict a class or estimate a score. An intelligent application must additionally manage data, state, decisions, external services, errors, and user interaction.

A practical system may contain:

- a user-interface component;
- a coordinator or workflow manager;
- specialized agents;
- databases and external services;
- validation and access-control rules;
- monitoring and evaluation components.

1.2 Main elements

Agent: A component with a role and decision policy.

Environment: The world or application state in which the agents operate.

Percept: Information available to an agent.

Action: An operation that changes state or produces an outcome.

Goal: A desired condition or performance objective.

Message: Information exchanged between agents.

Coordination: Mechanisms for ordering or reconciling agent actions.

1.3 Example

A shopping assistant may use a catalog agent to retrieve products, a recommendation agent to rank them, and an inventory agent to verify availability. A coordinator combines the results into a response.

A collection of functions is not automatically a MAS. Agent boundaries should represent meaningful responsibilities, decision policies, or independently evolving services.

2 Examples of Multiagent Systems

2.1 Applications in different domains

Domain	Agents	Objective
E-commerce	Catalog, recommendation, inventory, policy	Relevant product selection under constraints
Tourism	Preference, attraction, route, budget	Feasible personalized itineraries
Healthcare support	Intake, information, scheduling, preparation	Safe administrative and educational assistance
Smart buildings	Occupancy, lighting, HVAC, energy	Balance comfort and energy use
Traffic	Signal controllers, traffic monitors	Reduce congestion while respecting safety constraints
Education	Learner model, tutor, exercise selector	Personalized learning activities
Supply chain	Demand, inventory, procurement, delivery	Coordinate stock and logistics

2.2 Centralized coordination

A central coordinator receives a request, assigns tasks, collects outputs, and decides what to do next.

Advantages:

- straightforward workflow control;
- easier debugging and monitoring;
- explicit validation of final outputs.

Limitations include a possible bottleneck and dependence on the coordinator.

2.3 Distributed coordination

Agents make local decisions and coordinate through messages, events, negotiation, or shared state. This can improve autonomy and scalability but introduces challenges involving conflicts, consistency, and failures.

2.4 When is MAS useful?

A multiagent approach is particularly useful when a problem naturally decomposes into different responsibilities, when components use different tools or policies, or when coordination between decisions has measurable value.

Design principle: Do not introduce agents merely to increase the number of components. Every agent introduces communication, testing, and maintenance costs.

3 Concepts for Understanding Intelligent Agents

3.1 Agent perception and action

At time t, an agent receives a percept o_t, maintains internal state b_t if required, and chooses an action a_t.

A simple policy can be represented by

$$a_t = \pi(o_t).$$

A stateful policy may be represented as

$$a_t = \pi(b_t, o_t),$$

where the internal state summarizes relevant past observations.

A stochastic policy can be written as

$$\pi(a \mid o) = P(a_t = a \mid o_t = o).$$

3.2 PEAS task specification

PEAS stands for Performance measure, Environment, Actuators, and Sensors. It provides a structured way to describe an agent's task.

Example: tourism itinerary agent

- Performance: preference match, budget compliance, feasible travel time.
- Environment: attractions, transport, opening hours, user constraints.
- Actuators: propose, add, remove, or reorder stops.
- Sensors: preferences, map results, schedules, weather.

3.3 Rationality

A rational agent selects an action expected to perform well according to its objective, given its available information, knowledge, and computational resources.

Rationality does not imply perfect knowledge. Data may be incomplete, noisy, or outdated.

3.4 Important properties

- **Autonomy:** decisions are based on the agent's policy and state.
- **Reactivity:** the agent responds to changes.
- **Proactiveness:** the agent plans toward goals.
- **Social ability:** the agent communicates.
- **Learning:** the agent improves from data or feedback.

4 Environments, Goals, and Agent Architectures

4.1 Environment characteristics

An environment can be described along several dimensions:

- fully or partially observable;
- deterministic or stochastic;
- episodic or sequential;
- static or dynamic;
- discrete or continuous;
- single-agent or multiagent.

These distinctions affect how an agent should represent uncertainty, plan, and react to events.

4.2 Common architectures

Reactive agents use condition–action rules. They are often fast and interpretable.

Deliberative agents maintain a representation of states, goals, and possible plans.

Belief–Desire–Intention (BDI) agents distinguish beliefs about the world, desired outcomes, and committed plans.

Utility-based agents compare candidate actions using a utility function:

$$a^* = \arg\max_{a \in \mathcal{A}} U(a).$$

Learning agents use data to estimate predictions or improve decision policies.

Hybrid agents combine rules, planning, retrieval, learned models, and tools.

4.3 Recommended student architecture

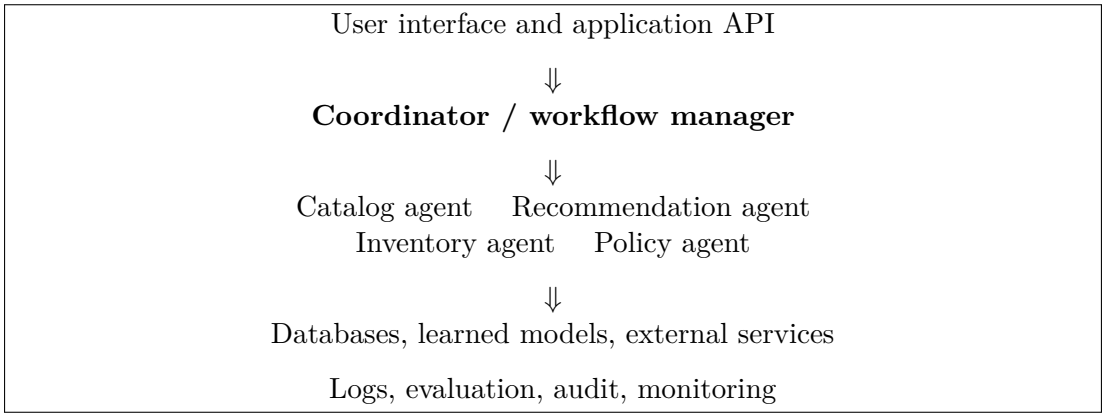
For an introductory MAS project, a hybrid architecture is usually practical:

1. rules enforce hard constraints and safety;
2. retrieval obtains current facts;
3. a learned model classifies or ranks candidates;
4. a coordinator controls the workflow;
5. a human reviews consequential decisions.

A language model can be included as a component, but natural-language generation alone does not provide reliable state management, authorization, or coordination.

5 Multiagent Architecture and Communication

5.1 Reference architecture



5.2 Message structure

Messages should have explicit schemas. A conceptual message may contain:

- message identifier;
- conversation or task identifier;
- sender and receiver;
- message type;
- structured payload;
- timestamp and optional timeout.

Example:

```
{
  "message_id": "m-001",
  "sender": "coordinator",
  "receiver": "catalog",
  "type": "SEARCH_PRODUCTS",
  "payload": {"query": "lightweight laptop"}
}
```

5.3 Communication patterns

- Request–response: one agent asks another for a result.
- Publish–subscribe: agents receive relevant events.
- Shared workspace: agents use controlled shared state.
- Negotiation: agents propose alternatives and resolve conflicts under explicit rules.

5.4 Coordination invariants

Examples of invariants that must remain true:

- A booking is not confirmed until the booking service confirms it.
- A recommendation must not imply verified stock when inventory has not been checked.
- A healthcare support model must not present an unverified prediction as a diagnosis.

6 Engineering Workflow: From Problem to MAS

A disciplined engineering workflow reduces complexity and makes the system testable.

1. Define the user problem and measurable success criteria.
2. Identify environment entities, data sources, and tools.
3. Decompose the task into coherent agent responsibilities.
4. Define message schemas and access permissions.
5. Specify ordering, timeouts, retries, and fallback policies.
6. Implement a minimal end-to-end working example.
7. Test normal inputs, edge cases, and failures.
8. Compare performance against a suitable baseline.
9. Deploy behind an interface and monitor outcomes.
10. Revise the system using evidence and feedback.

6.1 Agent specification template

For each agent, document:

- name and responsibility;
- input and output schemas;
- permitted tools and data;
- decision policy;
- expected failure modes;
- success metric and tests.

6.2 Why explicit contracts matter

An agent's output should be validated before it is consumed by another agent. For example, a recommendation agent should return a list of known product identifiers, not arbitrary product descriptions.

The coordinator should enforce workflow rules rather than blindly trusting the outputs of internal components.

6.3 Minimum viable implementation

Begin with two agents and one coordinator. Add more agents only when their responsibilities are distinct and their contribution can be evaluated.

7 Python Environment and a Basic Agent

The initial implementation uses the Python standard library. PyTorch is required only for the model extension.

Install Python 3.10 or later. In a terminal, check the environment and optionally install PyTorch:

```
python --version
python -m pip install torch
```

7.1 Message and agent interface

```
from dataclasses import dataclass
from typing import Any

@dataclass
class Message:
    sender: str
    receiver: str
    kind: str
    payload: dict[str, Any]

class Agent:
    def __init__(self, name: str):
        self.name = name

    def handle(self, message: Message) -> Message:
        raise NotImplementedError
```

The `Message` class defines a basic communication contract. Each agent implements its own handling policy.

7.2 Catalog agent

```
class CatalogAgent(Agent):
    def __init__(self, products):
        super().__init__("catalog")
        self.products = products

    def handle(self, message):
        query = message.payload.get(
            "query", ""
        ).strip().lower()

        def matches(item):
            fields = [
                item["name"],
                item["category"]
            ]
            fields += item.get("tags", [])
            return any(
                query in str(value).lower()
                for value in fields
            )

        items = [
            item for item in self.products
            if query and matches(item)
        ]

        return Message(
            sender=self.name,
            receiver=message.sender,
            kind="SEARCH_RESULTS",
```

```
            payload={"items": items}
        )
```

The catalog agent retrieves candidates. It does not rank them or decide whether they are affordable. Those tasks belong to other components.

8 Building an E-commerce Multiagent System

We construct a shopping assistant with a small in-memory catalog. This example runs without external APIs.

8.1 Product data

```
PRODUCTS = [
    {
        "id": "p1",
        "name": "Lightweight laptop",
        "category": "electronics",
        "price": 850,
        "tags": ["portable", "study"]
    },
    {
        "id": "p2",
        "name": "Wireless mouse",
        "category": "electronics",
        "price": 25,
        "tags": ["portable", "accessory"]
    },
    {
        "id": "p3",
        "name": "Running shoes",
        "category": "sports",
        "price": 70,
        "tags": ["fitness", "outdoor"]
    },
    {
        "id": "p4",
        "name": "Travel backpack",
        "category": "travel",
        "price": 55,
        "tags": ["portable", "travel"]
    }
]
```

8.2 Recommendation agent

```
class RecommendationAgent(Agent):
    def __init__(self):
        super().__init__("recommender")

    def handle(self, message):
        budget = float(
            message.payload.get("budget", 0)
        )
        items = message.payload.get("items", [])

        candidates = [
            item for item in items
            if item["price"] <= budget
        ]

        ranked = sorted(
            candidates,
            key=lambda item: (
                "portable" not in item["tags"],
                item["price"]
            )
        )

```

```
        return Message(
            self.name,
            message.sender,
            "RECOMMENDATIONS",
            {"items": ranked[:3]}
        )
```

The agent first applies the budget constraint, then ranks eligible items. A real recommender could additionally use user preferences, product attributes, or a trained model.

9 Coordinator, Execution, and Testing

9.1 Coordinator implementation

```
class Coordinator(Agent):
    def __init__(self, catalog, recommender):
        super().__init__("coordinator")
        self.catalog = catalog
        self.recommender = recommender

    def shop(self, query, budget):
        search_msg = Message(
            self.name, "catalog", "SEARCH",
            {"query": query}
        )
        search_result = self.catalog.handle(
            search_msg
        )

        rank_msg = Message(
            self.name, "recommender", "RANK",
            {
                "items": search_result.payload["items"],
                "budget": budget
            }
        )

        result = self.recommender.handle(rank_msg)
        return result.payload["items"]
```

9.2 Run the system

```
catalog = CatalogAgent(PRODUCTS)
recommender = RecommendationAgent()

system = Coordinator(catalog, recommender)

results = system.shop("portable", budget=100)

for product in results:
    print(product["name"], product["price"])
```

Expected products include the wireless mouse and travel backpack. The laptop is excluded because it exceeds the budget.

9.3 Testing requirements

Students should verify:

1. a matching tag retrieves the relevant item;
2. an item over budget is excluded;
3. an unknown query returns an empty result;
4. an empty query does not accidentally return everything;
5. the recommender only returns catalog candidates.

9.4 Extension: inventory agent

Add an inventory agent that returns stock information for each candidate. The coordinator must verify stock before claiming that a product is available.

If the inventory agent fails, the system should report that availability is unverified rather than inventing an answer.

10 Integrating PyTorch: Intent Classification

A trained classifier can help route user requests to an appropriate workflow. For example, requests may be assigned to e-commerce, tourism, or healthcare-support categories.

The following network demonstrates the mechanics of a PyTorch classifier. Its input consists of two numeric features, not raw natural language.

10.1 Model definition

```
import torch
from torch import nn

class IntentNet(nn.Module):
    def __init__(self, input_dim=2, n_classes=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, n_classes)
        )

    def forward(self, x):
        return self.net(x)

torch.manual_seed(42)

model = IntentNet()
model.eval()

x = torch.tensor(
    [[0.8, 0.2]],
    dtype=torch.float32
)

with torch.no_grad():
    logits = model(x)
    probabilities = torch.softmax(logits, dim=1)

labels = ["ecommerce", "tourism", "healthcare"]
prediction = int(probabilities.argmax(dim=1).item())

print("Predicted intent:", labels[prediction])
print("Probabilities:", probabilities.tolist())
```

10.2 Important limitation

The network is untrained. Its output is arbitrary and must not be used as a valid intent prediction. This example demonstrates tensor dimensions, model execution, and probability calculation only.

10.3 Training outline

Given a labeled training set `train_x` with shape $[N, 2]$ and integer labels `train_y` in $\{0, 1, 2\}$, the model can be trained as follows.

```
model = IntentNet()
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-3
)
criterion = nn.CrossEntropyLoss()

for epoch in range(50):
    model.train()
```

```
    logits = model(train_x)
    loss = criterion(logits, train_y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

A valid application also requires representative labeled data, a held-out test set, suitable features, and evaluation of classification errors.

11 Coordination, Failure Handling, and Monitoring

A coordinator is a workflow controller. It decides which agent to call, validates outputs, and handles missing or inconsistent information.

11.1 Failure scenarios

Typical failures include:

- an agent times out;
- a message has an invalid payload;
- a database returns no records;
- a service returns stale information;
- two agents produce incompatible results.

11.2 Basic fallback pattern

```python
def safe_call(agent, message, fallback):
    try:
        response = agent.handle(message)

        if not isinstance(response, Message):
            raise TypeError(
                "Invalid agent response"
            )

        if response.kind.endswith("_ERROR"):
            return fallback

        return response

    except (TimeoutError, TypeError, ValueError):
        # A production implementation should log
        # a structured error and correlation ID.
        return fallback
```

This simple example does not implement a real timeout. A production implementation should enforce deadlines at the process, task, or network-call level.

11.3 Metrics

Task completion rate: Fraction of requests that meet the task definition.

Latency: Median and 95th-percentile response time.

Coordination overhead: Messages and model/tool calls per request.

Quality: Relevance and constraint satisfaction.

Reliability: Failures, timeouts, and recovery success.

Safety: Unauthorized actions and unsupported claims.

11.4 Reproducibility

Record model and policy versions, message sequence, timestamps, and final outcomes. Avoid logging secrets or unnecessary personal data. A fixed random seed improves experimental repeatability but does not establish real-world validity.

12 Case Study A: E-commerce Shopping Assistant

12.1 Problem definition

A customer requests products that satisfy preferences, budget, and other constraints. The system should retrieve real candidates, rank them, verify relevant details, and explain the recommendations.

12.2 Agent roles

- **Intent agent:** classifies the request.
- **Catalog agent:** retrieves products.
- **Recommendation agent:** ranks candidates.
- **Inventory agent:** verifies stock.
- **Policy agent:** checks business constraints.
- **Coordinator:** manages the workflow.

12.3 Workflow

Customer request → intent classification → catalog retrieval → constraint filtering → ranking → inventory verification → response

12.4 Evaluation

Possible metrics include:

- Recall@k for retrieval;
- NDCG@k for ranking quality;
- budget-violation rate;
- unsupported-availability claims;
- task completion and latency.

Compare the MAS with a single-agent baseline using the same requests, catalog, and evaluation criteria.

12.5 Ethical considerations

Avoid manipulative ranking and undisclosed sponsored placement. Clearly distinguish relevance-based ranking from paid promotion. Do not place orders or charge payments without explicit user confirmation.

13 Case Study B: Tourism and Itinerary Planning

13.1 Problem definition

A traveler wants an itinerary based on dates, interests, budget, travel time, opening hours, and personal constraints. The itinerary must be both useful and feasible.

13.2 Agent roles

- Preference agent: collects interests and constraints.
- Attraction agent: retrieves candidate destinations.
- Route agent: estimates travel times and ordering.
- Budget agent: estimates costs.
- Context agent: retrieves relevant weather or event data.
- Coordinator: reconciles conflicts and produces the plan.

13.3 Basic mathematical formulation

Let $x_i \in \{0, 1\}$ indicate whether attraction i is selected. Let c_i denote its cost, t_i its visit duration, and T_{ij} the travel time between attractions.

A budget constraint is

$$\sum_{i=1}^{n} c_i x_i \leq B,$$

where B is the available budget.

A simplified time constraint is

$$\sum_{i=1}^{n} t_i x_i + \sum_{(i,j)\in R} T_{ij} \leq T_{\max},$$

where R is the set of consecutive itinerary transitions.

A real itinerary must additionally account for opening hours, waiting times, breaks, and time windows.

13.4 Coordination challenge

The attraction agent may recommend a site that is closed. The route agent may find an inefficient order, while the budget agent may reject the total cost.

The coordinator should revise the candidate set, relax a soft preference, or ask the user which constraint can change.

13.5 Evaluation

Measure feasibility, preference match, budget deviation, source freshness, itinerary diversity, and the number of clarification turns.

Opening hours and transport estimates should be verified against appropriate current sources.

14 Case Study C: Healthcare Support

14.1 Scope and safety boundary

A student prototype should focus on administrative or educational support rather than autonomous diagnosis or treatment. Examples include appointment preparation, clinic navigation, and organizing questions for a clinician.

14.2 Agent roles

- **Intake agent:** clarifies the administrative need.
- **Information agent:** retrieves approved educational material.
- **Scheduling agent:** queries appointment slots.
- **Preparation agent:** creates a neutral checklist.
- **Safety agent:** identifies cases requiring professional or emergency support.
- **Coordinator:** enforces permissions and workflow.

14.3 Example workflow

A user requests help preparing for a clinic visit.

1. The intake agent clarifies the purpose and logistics.
2. The information agent retrieves approved clinic material.
3. The preparation agent generates a checklist.
4. The user reviews the checklist.
5. Qualified healthcare personnel remain responsible for clinical decisions.

14.4 Safety requirements

- Minimize personal-data collection.
- Use consent, access controls, and secure storage.
- Label generated content and identify its source.
- Never fabricate appointments, test results, or records.
- Escalate urgent or high-risk requests appropriately.
- Require confirmation before booking or changing records.

14.5 Evaluation

Measure information correctness, task completion, privacy compliance, escalation sensitivity, false reassurance, and usability. The system must not present an unverified model output as a professional medical judgment.

15 Comparing the Three Applications

Dimension	E-commerce	Tourism	Healthcare support
Objective	Product relevance	Feasible itinerary	Safe administrative support
Authoritative data	Catalog, price, inventory	Opening hours, maps, costs	Approved content, clinic schedule
Typical conflict	Budget vs. preference	Interest vs. time	Convenience vs. safety
Key metric	Ranking and constraint violations	Feasibility and preference match	Correctness and escalation
Confirmation	Before purchase	Before booking or purchase	Before appointment or record change
Failure fallback	Show verified candidates	Offer revised itinerary	Stop risky automation and escalate

15.1 Cross-domain design principles

Despite different application requirements, all three systems benefit from:

- explicit agent responsibilities;
- structured communication;
- verification of important facts;
- measurable constraints;
- graceful failure handling;
- appropriate human confirmation.

The acceptable risk level differs by domain. A ranking error in a shopping recommendation and an unsafe healthcare response must not be evaluated using the same criteria.

15.2 Discussion

Ask students:

1. Which agent owns each constraint?
2. Which facts require verification?
3. What should happen when two agents disagree?
4. Which actions require explicit user confirmation?
5. How can the system communicate uncertainty?

16 Experimental Design and Evaluation

A working demonstration is necessary but insufficient. A student project should test whether the MAS improves a measurable property relative to a suitable baseline.

16.1 Baselines

B0: Single-agent pipeline.

B1: Rule-based multiagent system.

B2: Multiagent system with a learned component.

B3: Ablation without coordinator validation.

16.2 Experimental controls

All systems should use the same test requests, data sources, and task definitions. A held-out test set should be kept separate from training and model selection.

16.3 Suggested metrics

- task completion rate;
- domain-specific relevance or accuracy;
- constraint satisfaction rate;
- median and 95th-percentile latency;
- messages and tool calls per request;
- failure recovery rate;
- safety violations.

16.4 Results table template

System	Success	P95 latency	Messages	Violations
B0: single agent	–	–	–	–
B1: rule-based MAS	–	–	–	–
B2: learned MAS	–	–	–	–
B3: no validation	–	–	–	–

Students should populate the table with measured results. Do not invent results or treat a toy demonstration as evidence of deployment readiness.

17 Laboratory Exercises and Deliverables

Exercise 1: Specify the agents

Choose one application. Write a PEAS specification and define at least three agents. For each agent, document its inputs, outputs, permissions, and success metrics.

Exercise 2: Extend the shopping system

Run the baseline Python code and add an inventory, policy, or preference agent. Update the coordinator and test the complete workflow.

Exercise 3: Integrate PyTorch

Create a labeled dataset for three request categories. Train the intent classifier and report accuracy and per-class precision and recall on a held-out test set.

Use a confidence threshold or uncertainty rule to decide when the system should request clarification rather than route a request automatically.

Exercise 4: Failure injection

Simulate:

1. an agent timeout;
2. an empty search result;
3. a malformed message;
4. stale availability information.

   Show that the system handles each case safely.

Exercise 5: Compare architectures

Compare a single-agent baseline with the MAS. Discuss whether the additional coordination improves quality enough to justify its latency and complexity.

Required deliverables

- architecture diagram and agent specifications;
- runnable Python source and README;
- model training and evaluation code, if applicable;
- at least eight tests, including three failure tests;
- results table and discussion of limitations;
- short presentation or demonstration.

18 Assessment Rubric and Common Mistakes

18.1 Suggested assessment rubric

Criterion	Points	Evidence
Problem definition and PEAS	10	Clear objective and environment
Agent decomposition and contracts	15	Roles, schemas, permissions
Implementation and reproducibility	20	Runnable code and instructions
Coordination and failure handling	15	Validation, timeouts, fallbacks
PyTorch/model evaluation	15	Valid data split and metrics
Testing and comparative analysis	15	Tests, baselines, measured results
Ethics and presentation	10	Privacy, safety, clear explanation
Total	100	

18.2 Common mistakes

- Calling several ordinary functions “agents” without meaningful independent responsibilities.
- Using an untrained neural network and treating its predictions as valid.
- Allowing agents to invent facts absent from the data.
- Ignoring timeouts, duplicate actions, or malformed data.
- Evaluating only accuracy while ignoring safety and latency.
- Using personal data without appropriate justification and safeguards.
- Reporting improvements without a fair baseline.

18.3 Final project suggestion

Students may build a tourism planner that combines preference elicitation, attraction retrieval, route planning, and budget constraints. They can compare a rule-based system with a learned ranking component and evaluate itinerary feasibility.

The same framework can be adapted to shopping or healthcare administration.

19 Reference Architecture and Further Reading

19.1 Final design checklist

Before presenting a multiagent system, verify that:

1. the problem and success criteria are explicit;
2. each agent has a coherent responsibility;
3. inputs and outputs follow defined schemas;
4. permissions and access to data are controlled;
5. the coordinator validates results;
6. external facts are verified when freshness matters;
7. learned components are trained and evaluated;
8. tests cover normal, boundary, and failure cases;
9. logs support debugging without unnecessary personal data;
10. consequential actions require suitable confirmation.

19.2 Foundational references

1. Russell, S. J., and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson. See the chapters on intelligent agents and multiagent decision making.
2. Wooldridge, M. *An Introduction to MultiAgent Systems*. Wiley.
3. Shoham, Y., and Leyton-Brown, K. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press.
4. Sutton, R. S., and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press.
5. PyTorch documentation. `https://pytorch.org/docs/stable/`

19.3 Conclusion

A multiagent system is an engineered collaboration among decision-making components. Strong implementations combine clear roles, structured communication, grounded information, measurable evaluation, safe failure handling, and human-centered workflows.

The goal is not to maximize the number of agents. It is to design a system in which coordination produces measurable benefits over a simpler alternative.

End of tutorial